

Event Bubbling & Event Capturing

How events travel through the DOM — event delegation, the target element, and the two propagation phases.

HTML

CSS

Event Bubbling

Event Capturing

Delegation

PART 1 — EVENT BUBBLING

1 Complete HTML + CSS Code

Your bubbling HTML contains a `<ul>` with five `<li>` elements.

```
index.html

<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>

    <style>
      * {
        padding: 0;
        margin: 0;
        font-family:
          "Gill Sans", "Gill Sans MT", Calibri, "Trebuchet MS", sans-serif;
        box-sizing: border-box;
      }

      body {
        padding: 30px;
      }

      #main {
        width: 100%;
        height: 100%;
        background-color: #322;
      }

      #nav {
        display: flex;
        align-items: center;
        justify-content: space-around;
        width: 100%;
        height: 100px;
        background-color: #444;
      }

      a {
        color: white;
      }
    </style>
  </head>
  <body>
    <div id="main">
      <div id="nav">
        <a href="#"></a>
      </div>
    </div>
  </body>
</html>
```

```

    ul {
      cursor: pointer;
    }

    button {
      padding: 12px 22px;
      background-color: red;
      border-radius: 5px;
    }

    .lt {
      text-decoration: line-through;
    }
  </style>
</head>

<body>
  <ul>
    <li>Apple</li>
    <li>Guava</li>
    <li>Grapes</li>
    <li>Banana</li>
    <li>Mango</li>
  </ul>

  <script src="bubbling.js"></script>
</body>
</html>

```

Important HTML

```

<ul>
  <li>Apple</li>
  <li>Guava</li>
  <li>Grapes</li>
  <li>Banana</li>
  <li>Mango</li>
</ul>

```

There are **5 elements**, but instead of adding 5 separate event listeners, you put **one listener on the parent **.

That's where event bubbling becomes useful.

2 Complete JavaScript Code

```

//--event bubbling
// main div
//   div
//     links
//     button

// document.querySelector("#nav").addEventListener("click", function () {
//   alert("clicked");
// });

// we got 5 li s so we will put listener on ul parent using event bubbling

let ul = document.querySelector("ul");

ul.addEventListener("click", function (details) {

```

bubbling.js

```
// details.target.style.textDecoration = "line-through";
// details.target.classList.add("lt");

details.target.classList.toggle("lt");
});

// let a = document.querySelector(".a");
// let b = document.querySelector(".b");
// let c = document.querySelector(".c");
// let btn = document.querySelector("button");

// btn.addEventListener("click", function () {
//   console.log("button clicked");
// });

// c.addEventListener("click", function () {
//   console.log("c clicked");
// });

// b.addEventListener("click", function () {
//   console.log("b clicked");
// });

// a.addEventListener("click", function () {
//   console.log("a clicked");
// });

// event flow phase 2
```

3 Event Bubbling

When an event happens on a child element, it **bubbles upward toward its parents**.

```
li → ul → body → html → document
```

If you click an `<li>`, the event can reach the `<ul>`.

4 event.target

```
details.target.classList.toggle("lt");
```

target tells you the actual element on which the event occurred.

So when you click `<li>Apple</li>`, then **details.target** refers to that `<li>`.

5 Event Delegation

Instead of doing this for every `<li>`:

```
li1.addEventListener("click", ...);
li2.addEventListener("click", ...);
li3.addEventListener("click", ...);
li4.addEventListener("click", ...);
li5.addEventListener("click", ...);
```

you use the parent:

```
ul.addEventListener("click", function (details) {  
  details.target.classList.toggle("lt");  
});
```

Because of **event bubbling**, the click reaches the `<ul>`. This technique is called **event delegation**.

6 `classList.toggle()`

```
details.target.classList.toggle("lt");
```

The CSS class:

```
.lt {  
  text-decoration: line-through;  
}
```

Class absent → adds it

Class present → removes it

So clicking a fruit item toggles the strikethrough. Click again and it disappears:

Apple
Guava
Grapes
Banana
Mango

PART 2 — EVENT CAPTURING

7 Complete HTML + CSS Code

Your capturing HTML creates a nested structure: `.a` → `.b` → `.c` → `button`.

```
<!DOCTYPE html>  
<html lang="en">  
  <head>  
    <meta charset="UTF-8" />  
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />  
    <title>Document</title>  
  
    <style>  
      * {  
        margin: 0;  
        padding: 0;  
        box-sizing: border-box;
```

index.html

```

    }

    body {
      background: #111;
    }

    .a {
      width: 85%;
      height: 345px;
      margin: 14px;
      background: #3b2c2c;
    }

    .b {
      width: 80%;
      height: 277px;
      background: darkgreen;
    }

    .c {
      width: 80%;
      height: 80%;
      background: #75182a;
    }

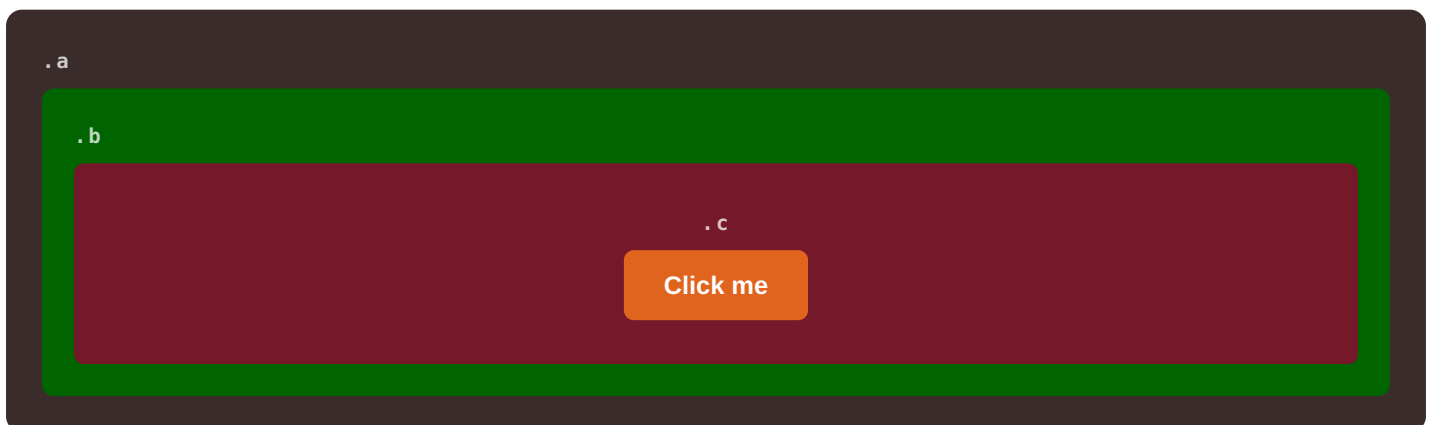
    button {
      margin: 2px;
    }
  </style>
</head>

<body>
  <div class="a">
    <div class="b">
      <div class="c">
        <button>Click me</button>
      </div>
    </div>
  </div>

  <script src="capturing.js"></script>
</body>
</html>

```

Structure



8 Complete JavaScript Code



```
// whenever we raise an event the event flow or propagation has two phases
// phase 1 : event goes downwards from the top level element
// phase 2: event goes from raised element to parent

// 1 occurs first

let a = document.querySelector(".a");
let b = document.querySelector(".b");
let c = document.querySelector(".c");
let btn = document.querySelector("button");

btn.addEventListener("click", function () {
  console.log("button clicked");
});

c.addEventListener(
  "click",
  function () {
    console.log("c clicked");
  },
  true,
);

b.addEventListener("click", function () {
  console.log("b clicked");
});

a.addEventListener(
  "click",
  function () {
    console.log("a clicked");
  },
  true,
);

// capture phase >>>> bubbling phase
// bubbling goes first normally when capture phase is not enabled
// capture phase is enabled by writing true
```

9 Event Flow

When you click the button, the event travels through the DOM. There are **two main phases**:

- | | |
|---------------------|----------------|
| 1. Capturing | Parent → Child |
| 2. Bubbling | Child → Parent |

10 true Enables Capturing

Normally:

```
element.addEventListener("click", function () {
  // ...
});
```

uses the **bubbling phase**. To run the listener during the capturing phase:

```
element.addEventListener(  
  "click",  
  function () {  
    // ...  
  },  
  true,  
);
```

The third argument `true` enables **capture mode**.

11 Your Example

Your structure is `.a → .b → .c → button`. You have:

a	→ true (capture)
c	→ true (capture)
b	→ normal (bubble)
button	→ normal (bubble)

So when the button is clicked:

Capturing	a → c
Bubbling	button → b

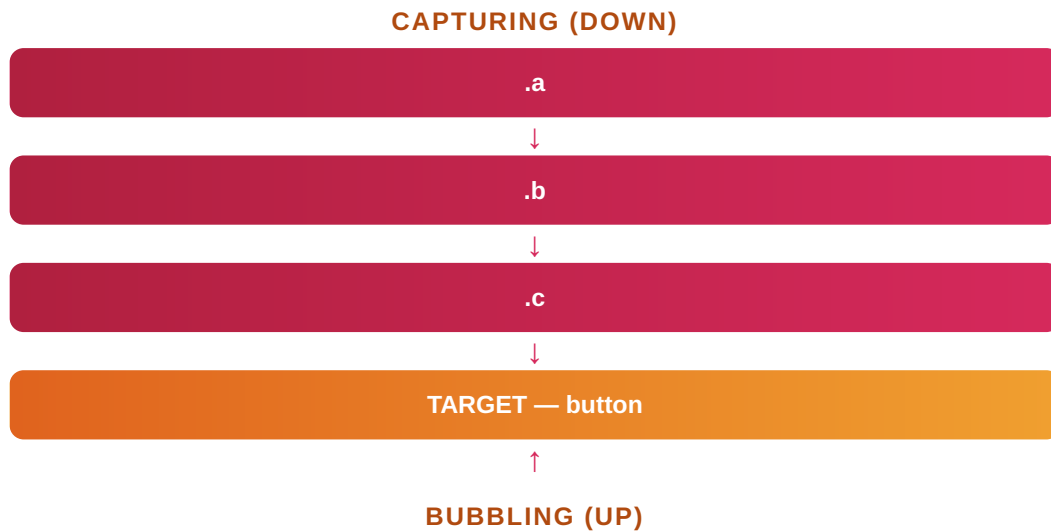
Therefore the console output is:

1. a clicked
2. c clicked
3. button clicked
4. b clicked

12 Bubbling vs Capturing

	Capturing	Bubbling
Direction	Parent → Child	Child → Parent
Default?	No	Yes
Enabled with	true	Normal listener
Example	a → c → button	button → c → a

13 Event Flow — Quick Diagram



The event first travels **down toward the target**, reaches the target, then travels **back upward**.

14 Quick Revision

Event bubbling

```
parent.addEventListener("click", function (event) {  
  console.log(event.target);  
});
```

Child event can reach the parent.

Event delegation

Use **one parent listener** instead of multiple child listeners.

```
ul.addEventListener("click", function (details) {  
  details.target.classList.toggle("lt");  
});
```

Capturing

```
element.addEventListener("click", function () {  
  // code  
}, true);
```

`true` → capturing phase.

🎯 CORE CONCEPTS

From these two files: `event.target`, event propagation, bubbling, capturing, event delegation, `classList.toggle()`, and the third `addEventListener()` argument (`true`).

✨ Bubbling lets one parent listener handle many children — capturing lets you intercept the click before it even gets there.